

İŞLETİM SİSTEMLERİ

Bölüm 2

Süreçler ve İş Parçacıkları

Processes and Threads

Modern Tanenbaum — İşletim Sistemleri kitabı Bölüm 2 temel alınarak hazırlanmıştır.

1. Süreçler (Processes)

İşletim sistemlerinin sunduğu en temel soyutlamalardan biri **süreç (process)** kavramıdır. Bir süreç, çalışmakta olan bir programın soyutlamasıdır: program sayacı (program counter), yazmaçlar (registers) ve değişkenlerin anlık değerlerini içerir. Kavramsal olarak her sürecin kendi sanal CPU'su vardır; gerçekte ise CPU süreçler arasında hızla geçiş yapar (multiprogramming / çoklu programlama).

■ Tanım

Süreç = Çalışan programın bir örneği (instance). Program ise disk üzerinde pasif olarak bekleyen bir varlıktır.

1.1 Süreç Modeli

Multiprogramming modelinde bir CPU'nun N program arasında hızla geçiş yaptığını düşünürüz. Her programın kendi mantıksal program sayacı vardır. Fiziksel program sayacı ise yalnızca o anda çalışan sürece ait değeri tutar; süreç değişiminde eski değer bellekte saklanır, yenisi yüklenir.

Kavram	Açıklama
Program	Disk üzerindeki pasif talimat dizisi (tarif)
Süreç (Process)	Çalışmakta olan program örneği (pişirme eylemi)
İş Parçacığı (Thread)	Süreç içindeki bağımsız yürütme akışı

Analoji: Aşçının elindeki tarif = program; aşçının gerçekleştirdiği pişirme eylemi = süreç. Aşçı araya giren bir olay için (arı sokması) farklı bir tarife (ilk yardım kitabı) geçerse, bu farklı öncelikte bir süreç geçiştir.

⚠ Dikkat

Aynı program iki kez çalıştırıldığında iki ayrı süreç oluşur. Paylaşılan kod segmenti olsa da bunlar bağımsız süreçlerdir.

1.2 Süreç Oluşturma (Process Creation)

Süreçlerin oluşturulmasına yol açan dört temel olay vardır:

- **1. Sistem başlatma (System initialization):** Arka plan süreçleri (daemon) oluşturulur.
- **2. Çalışan sürecin sistem çağrısı:** Mevcut süreç yeni bir süreç yaratır.
- **3. Kullanıcı isteği:** Komut satırına komut yazılması veya ikona tıklanması.
- **4. Toplu iş başlatma (Batch job):** Büyük sistemlerde giriş kuyruğundan iş alınması.

İşletim Sistemi	Süreç Oluşturma Çağrısı	Açıklama
UNIX/Linux	fork()	Ebeveyn sürecin birebir klonunu oluşturur; ardından execve() ile program değiştirilir.
Windows	CreateProcess()	10 parametrelili tek çağrıyla hem süreç oluşturur hem program yükler.

```
/* UNIX: fork + execve kalıbı */
pid_t pid = fork();
if (pid == 0) {
/* Çocuk süreç */
execve("/bin/sort", argv, envp);
} else {
/* Ebeveyn süreç – çocuğu bekleyebilir */
wait(NULL);
}
```

□ fork() Sonrası

UNIX'te fork() sonrasında ebeveyn ve çocuk iki ayrı adres alanına sahiptir. Çağrı öncesi bellek yazma sırasında kopyalanır (copy-on-write). fork() ebeveyne çocuğun PID'ini, çocuğa 0 döndürür.

1.3 Süreç Sonlandırma (Process Termination)

Süreçler dört koşulda sona erer:

- **1. Normal çıkış (gönüllü):** exit() çağrısı — iş tamamlandı.
- **2. Hata çıkışı (gönüllü):** Geçersiz girdi gibi beklenen hata durumu.
- **3. Ölümcül hata (zorunlu):** Sıfıra bölme, geçersiz bellek erişimi vb.
- **4. Başka süreç tarafından öldürülme (zorunlu):** UNIX'te kill(), Windows'ta TerminateProcess().

1.4 Süreç Hiyerarşisi

UNIX'te her süreç yalnızca bir ebeveyni olan ağaç yapısındadır. Tüm süreçler önyükleme sırasında oluşturulan **init** sürecinin alt kümesidir. Klavye sinyali (CTRL-C) tüm süreç grubuna gönderilir.

Windows'ta süreç hiyerarşisi yoktur; ebeveyn çocuğa handle (tanıtıcı) verir, bu handle devredilerek hiyerarşi bozulabilir.

1.5 Süreç Durumları (Process States)

Her süreç üç temel durumdan birinde bulunur:

Durum	Açıklama	Geçiş
Running (Çalışıyor)	CPU'yu aktif olarak kullanan tek süreç	Dört geçiş yönü var
Ready (Hazır)	Çalışmaya hazır, CPU bekliyor	Zamanlayıcı seçimiyle Running'e geçer
Blocked (Engelli)	Bir olayı bekliyor (I/O, sinyal...)	Olay gerçekleşince Ready'e döner

Geçişler: (1) Süreç girdi bekler → Blocked; (2) Zamanlayıcı başka süreci seçer → Ready; (3) Zamanlayıcı bu süreci seçer → Running; (4) Beklenen olay gerçekleşir → Ready.

Önemli Fark

Ready ile Blocked arasındaki fark: Ready durumundaki süreç CPU verilse çalışır; Blocked durumundaki süreç CPU verilse bile çalışamaz, harici bir olayı bekler.

1.6 Süreçlerin Uygulanması (Process Table)

İşletim sistemi her süreç için bir **süreç tablosu (process table)** girişi tutar. Bu giriş, süreç yeniden başlatılabilmesi için gereken tüm bilgileri içerir:

Süreç Yönetimi	Bellek Yönetimi	Dosya Yönetimi
Yazmaçlar, PC, PSW	Metin segmenti göstericisi	Kök dizin
Yığıt göstericisi	Veri segmenti göstericisi	Çalışma dizini
Süreç durumu, öncelik	Yığıt segmenti göstericisi	Dosya tanımlayıcıları
PID, ebeveyn PID	Kullanıcı ID, Grup ID	Sinyal işleyiciler

Kesme oluştuğunda donanım yazmaçları ve PC'yi yığına iter; ISR (Interrupt Service Routine) kaydedip süreç tablosuna yazar. Ardından zamanlayıcı çalışacak süreci seçer.

1.7 Multiprogramming Modeli

Bir süreç zamanının p oranında G/Ç beklediğini varsayalım. Bellekte n süreç varsa tüm süreçlerin aynı anda G/Ç bekleme olasılığı p^n 'dir. CPU kullanımı:

$$\text{CPU kullanımı} = 1 - p^n$$

Örnek: $p = 0.8$ (yüzde 80 G/Ç bekleme), $n = 10$ süreç $\rightarrow$ CPU kullanımı $= 1 - 0.8^{10} \approx 0.89$ (yaklaşık yüzde 89).

G/Ç Bekleme (p)	$n=1$	$n=3$	$n=5$	$n=10$
%20 (0.20)	%80	%99+	%99+	%99+
%50 (0.50)	%50	%87.5	%96.9	%99.9
%80 (0.80)	%20	%49	%67	%89

Model Sınırlılığı

Bu model süreçlerin bağımsız olduğunu varsayar; kuyruk teorisi daha doğru sonuçlar verir. Yine de genel fikri doğru aktarır: çok programlama, boşta bekleyen CPU'yu azaltır.

2. İş Parçacıkları (Threads)

Geleneksel süreçler tek bir kontrol akışına (thread) sahiptir. Ancak bazı uygulamalar aynı adres alanında birden fazla eş zamanlı etkinlik yürütmekten yararlanır. İş parçacığı (thread), bir süreç içindeki bağımsız yürütme akışdır.

Özellik	Süreç (Process)	İş Parçacığı (Thread)
Adres Alanı	Kendi özel alanı	Sürecin alanını paylaşır
Oluşturma maliyeti	Yüksek (10-100x)	Düşük
İletişim	IPC gerekli (karmaşık)	Paylaşılan bellek (kolay)
Hata izolasyonu	Birbirinden bağımsız	Bir thread crash → tüm süreç etkilenebilir
Zamanlama birimi	Bağımsız	Kernel thread → bağımsız; User thread → süreç kuantumunu paylaşır

2.1 Thread Kullanım Nedenleri

Paralellik + Paylaşılan Bellek	Birden fazla etkinlik aynı veriyi paylaşarak çalışır. Ayrı süreçlerle bunu yapmak mümkün değildir (farklı adres alanları).
Hafif Oluşturma	Thread oluşturma, süreç oluşturmaya göre 10-100 kat daha hızlıdır. Thread sayısı dinamik ve hızlı değişen uygulamalar için idealdir.
I/O ile Hesaplamayı Örtüştürme	Bir thread G/Ç beklerken başka bir thread CPU'da hesaplama yapabilir.
Çok Çekirdekli Sistemler	Gerçek paralellik: birden fazla thread farklı CPU çekirdeklerinde eş zamanlı çalışır.

Örnek: Çok Thread'li Kelime İşlemci

800 sayfalık bir belgede 1. sayfadan cümle silindiğinde 600. sayfaya atlamak istenirse, tek thread'li bir uygulama tüm sayfaları yeniden biçimlendirmek için uzun süre bekletir:

- **Thread 1 (kullanıcı etkileşimi):** Klavye/fare olaylarını işler.
- **Thread 2 (biçimlendirme):** Arka planda belgeyi yeniden biçimlendirir.
- **Thread 3 (otomatik kaydetme):** Belirli aralıklarla diske yazar.

✓ **Avantaj** Üç thread sayesinde kullanıcı sorunsuz çalışmaya devam eder. Üç ayrı süreç kullanılamaz çünkü hepsinin aynı belge verisine erişmesi gerekir.

Örnek: Çok Thread'li Web Sunucu

İstek geldiğinde dispatcher thread boş bir worker thread'e işi atar; worker önbellekte (cache) yoksa diskten okur ve bu sırada bloklanır — diğer thread'ler çalışmaya devam eder.

Model	Paralellik	Sistem Çağrıları	Zorluk
Çok thread'li	Evet	Bloklayan (basit)	Orta
Tek thread'li	Hayır	Bloklayan	Düşük
Olay tabanlı (FSM)	Evet	Bloklayan değil	Yüksek

2.2 Klasik Thread Modeli

Her thread şu bilgilere sahiptir: Program sayacı, yazmaçlar, yığıt (stack) ve durum. Süreç kaynaklarını (adres alanı, açık dosyalar, sinyaller) tüm thread'ler paylaşır.

Süreç başına (Per-Process)	Thread başına (Per-Thread)
Adres alanı	Program sayacı (PC)
Global değişkenler	Yazmaçlar
Açık dosyalar	Yığıt (Stack)
Çocuk süreçler	Durum (Running/Ready/Blocked)
Sinyaller ve işleyiciler	

Her thread kendi yığıtına sahiptir çünkü farklı prosedürleri çağırır, dolayısıyla farklı yürütme geçmişine sahiptir. Çok thread'li süreçlerde thread'ler başlangıçta tek thread'le başlar; thread_create() ile yeni thread açılır.

2.3 POSIX Threads (Pthreads)

IEEE 1003.1c standardı, taşınabilir thread programlama için **Pthreads** API'sini tanımlar. Tüm UNIX sistemleri destekler.

Fonksiyon	Açıklama
pthread_create()	Yeni thread oluşturur
pthread_exit()	Çağırın thread'i sonlandırır
pthread_join()	Belirtilen thread'in bitmesini bekler
pthread_yield()	CPU'yu başka thread'e bırakır
pthread_attr_init()	Thread öznelik yapısı oluşturur
pthread_attr_destroy()	Öznelik yapısını serbest bırakır

```
#include
#include
#define THREAD_COUNT 10
void *merhaba(void *tid) {
printf("Merhaba thread %d\n", (int)tid);
pthread_exit(NULL);
}
int main() {
pthread_t t[THREAD_COUNT];
for (int i = 0; i < THREAD_COUNT; i++)
pthread_create(&t[i], NULL, merhaba, (void*)i);
/* main biter, thread'ler de biter (exit(NULL)) */
exit(NULL);
}
```

2.4 Thread Uygulama Yerleri

User-Level Thread'ler

Kernel thread'lerden habersizdir; thread yönetimi bir kütüphane (run-time system) tarafından yapılır. Her süreç kendi thread tablosunu tutar.

✓ **Avantajlar** Kernel çağrısı gerekmez → thread geçişi çok hızlı (birkaç talimat). Uygulamaya özel zamanlama algoritması kullanılabilir. Mevcut işletim sistemleri üzerinde çalışabilir.

✗ **Dezavantajlar** Bloklayan sistem çağrısı tüm süreci durdurur. Sayfa hatası (page fault) tüm thread'leri bloklar. Round-robin zamanlama için saat kesmesi yoktur → thread sonsuza çalışabilir.

Kernel-Level Thread'ler

Kernel her thread'i bilir ve doğrudan thread tablosunda tutar. Bir thread bloklarsa aynı sürecin başka thread'ini çalıştırabilir.

✓ **Avantajlar** Bir thread bloklarsa diğerleri çalışmaya devam eder. Sayfa hatalarında başka thread seçilebilir. Gerçek çok çekirdekli paralellik mümkündür.

✗ **Dezavantajlar** Thread oluşturma/yok etme sistem çağrısı gerektirir → daha yavaş. Yüksek sayıda thread'te kernel kaynaklarını tüketir.

Hibrit Uygulama

Kernel-level thread'ler üzerine user-level thread'ler çoklama (multiplexing): programcı kaç kernel thread kullanacağını ve her birine kaç user thread atanacağını belirler. En esnek yaklaşımdır.

3. Senkronizasyon ve Süreçlerarası İletişim (IPC)

Birden fazla süreç/thread paylaşılan veriye aynı anda eriştiğinde üç temel sorun ortaya çıkar:

- **1. Yarış koşulu (race condition):** Nihai sonuç çalışma sırasına bağlıdır.
- **2. Karşılıklı dışlama (mutual exclusion):** Kritik bölgede aynı anda yalnızca bir süreç olmalı.
- **3. Bağımlılık sıralaması:** A verisi üretirken B beklemeli, veri hazır olunca B devam etmeli.

3.1 Yarış Koşulu (Race Condition)

İki süreç aynı anda yazdırma kuyruk değişkenini (*in*) okuduğunda her ikisi de aynı yuvayı kullanabilir. Sonuçta bir dosya adı kaybolur, yazıcı demon yanlış çalışır. Bu tür durumlara **yarış koşulu** denir.

```
/* KÖTÜ: Atomik olmayan okuma-yazma */
int next_free = in; /* A süreci okuyor: 7 */
/* -- Zamanlayıcı A'yı keser, B çalışır -- */
int next_free = in; /* B süreci de okuyor: 7 */
/* Her iki süreç de 7. yuvaya yazıyor! */
```

3.2 Kritik Bölge (Critical Region / Section)

Yarış koşulunu önlemek için paylaşılan veriye erişilen kod parçasının (**kritik bölge**) aynı anda yalnızca bir süreç tarafından yürütülmesi sağlanmalıdır. İyi bir çözüm için dört koşul gereklidir:

- İki süreç aynı anda kritik bölgesinde bulunamaz.
- CPU sayısı veya hızı hakkında varsayım yapılmaz.
- Kritik bölgesi dışındaki bir süreç başkasını engelleyemez.
- Hiçbir süreç kritik bölgesine girmek için sonsuza beklemek zorunda kalmaz.

3.3 Meşgul Bekleme ile Karşılıklı Dışlama

Kesme Devre Dışı Bırakma

Tek işlemcide kritik bölgeye girerken tüm kesmeleri devre dışı bırakmak, kritik bölgeden çıkarken yeniden etkinleştirmek mümkündür.

✗ **Sakıncası** Kullanıcı programlarına bu yetki verilmesi tehlikelidir. Çok işlemcili sistemlerde etkisizdir (diğer CPU'lar etkilenmez).

Peterson Çözümü

turn ve interested dizileri kullanılarak yazılımla karşılıklı dışlama sağlanır. Donanım desteği gerektirmez, N=2 süreç için doğru çalışır.

```

int turn;
int interested[2]; /* Başlangıç: {0, 0} */
void enter_region(int proc) {
    int other = 1 - proc;
    interested[proc] = 1; /* İlgi bildir */
    turn = proc; /* Sıra bende */
    while (turn == proc && interested[other]) ; /* Bekle */
}
void leave_region(int proc) {
    interested[proc] = 0; /* Artık ilgi yok */
}

```

✓ **Garanti** Her iki süreç aynı anda enter_region çağırırsa, turn'ün son yazılanı geçerlidir; diğeri while döngüsünde bekler. Dört koşul sağlanır.

TSL / XCHG Talimatı

TSL RX, LOCK (Test and Set Lock): Bellek sözcüğünü atomik olarak okur ve sıfır dışı bir değer yazar. Okuma-yazma arasında başka CPU erişemez.

```

enter_region:
    TSL REGISTER, LOCK ; lock'u yaz, eski değeri kaydet
    CMP REGISTER, #0 ; sıfır mıydı?
    JNE enter_region ; değilse tekrar dene (busy waiting)
    RET
leave_region:
    MOVE LOCK, #0 ; kilidi aç
    RET

```

⚠ **Meşgul Bekleme (Busy Waiting)** Yukarıdaki çözümlerin tümünde süreç kilidi beklerken CPU'yu boşa harcar. Yüksek öncelikli süreç + düşük öncelikli süreç senaryosunda öncelik tersine dönmesi (priority inversion) oluşabilir.

3.4 Uyku ve Uyandırma (Sleep / Wakeup)

Meşgul bekleme yerine bloklanma: sleep() çağırın süreç uyur; başkası wakeup() ile uyandırır. Ancak **kaybolan uyandırma sinyali (lost wakeup)** problemi çözülmezse iki süreç de sonsuza uyuya kalabilir.

Üretici-tüketici örneğinde: count = 0, tüketici kontrol edip uyumak üzereyken zamanlayıcı üreticiye geçerse ve üretici wakeup() gönderirse, tüketici henüz uykuya geçmemiş olduğundan sinyal kaybolur. Her ikisi de uyur → kilitlenme.

3.5 Semaforlar (Semaphores)

Dijkstra (1965) tarafından önerilen semafor, kaybolan uyandırmaları tamsayı sayacıyla tutar:

- **down(s) [P, wait]:** $s > 0$ ise azalt ve devam et; $s = 0$ ise bloklan.
- **up(s) [V, signal]:** s artır; bekleyen süreç varsa birini uyandır.
- Her iki işlem **atomik (indivisible)** olmalıdır.

```
/* Üretici-Tüketici – Semaphor Çözümü */
semaphore mutex = 1; /* Kritik bölge kilidi */
semaphore empty = N; /* Boş yuva sayısı */
semaphore full = 0; /* Dolu yuva sayısı */

void producer() {
while (1) {
item = produce();
down(&empty); /* Boş yuva bekle */
down(&mutex); /* Kritik bölgeye gir */
insert(item);
up(&mutex); /* Kritik bölgeden çık */
up(&full); /* Dolu yuvayı bildir */
}
}

void consumer() {
while (1) {
down(&full); /* Dolu yuva bekle */
down(&mutex);
item = remove();
up(&mutex);
up(&empty); /* Boş yuvayı bildir */
consume(item);
}
}
```

İki Kullanım Şekli

mutex semaforu: karşılıklı dışlama (1 başlar). full/empty semaforları: senkronizasyon (üretici dolu tamponu doldurursa, tüketici boş tamponu boşaltırsa wait/sleep).

3.6 Mutex

Semaforun sadece 0/1 değer aldığı basit hali. Kullanıcı alanında TSL veya XCHG ile verimli şekilde uygulanabilir. Kernel çağrısı gerekmez.

mutex_lock: Kilitliyse `thread_yield()` çağırarak CPU'yu bırakır (busy waiting değil). Böylece aynı sürecin başka thread'i çalışabilir.

Futex (Fast User-Space Mutex — Linux): Çekişme yoksa tamamen kullanıcı alanında çalışır; çekişme varsa kernel wait-queue kullanır. En iyi iki dünya birleştirilir.

3.7 Monitörler (Monitors)

Semaforlarda sıra hataları kilitlenmeye (deadlock) yol açabilir. Brinch Hansen ve Hoare (1974), daha güvenli yüksek seviyeli ilkel olarak **monitör** kavramını önerdi.

Monitör: prosedürleri, değişkenleri ve veri yapılarını bir araya getiren özel modül. **Aynı anda yalnızca bir süreç monitör içinde aktif olabilir** — bunu derleyici garanti eder.

Koşul değişkenleri (condition variables) ile **wait** ve **signal** işlemleri eklenir. Java'daki **synchronized** anahtar kelimesi monitörü uygular.

⚠ **Sınırlılık** Monitörler dil yapısıdır; C/C++ gibi dillerde doğrudan yoktur. Dağıtık sistemlerde (farklı makineler) işe yaramaz.

3.8 Mesaj Geçişi (Message Passing)

Paylaşılan bellek olmayan sistemlerde kullanılabilir. İki ilkel:

- **send(hedef, &mesaj);**: Hedefte mesaj gönderir.
- **receive(kaynak, &mesaj);**: Kaynaktan mesaj alır; yoksa bloklanır.

Tasarım sorunları: Kayıp mesajlar (ACK + sıra numarası ile çözülür), adres şeması (mailbox), kimlik doğrulama, performans.

3.9 Bariyer (Barrier)

Bir grup süreç, bir aşama tamamlanmadan bir sonrakine geçemez. Tüm süreçler bariyere ulaşana kadar bekler, ardından hepsi serbest kalır. Paralel matris hesaplamalarında yaygın kullanılır.

3.10 Öncelik Tersine Dönmesi (Priority Inversion)

Düşük öncelikli süreç mutex tutarken, yüksek öncelikli süreç aynı mutex'i isterse bloklanır. Orta öncelikli süreç çalışmaya devam eder; yüksek öncelikli süreç sanki daha az öncelikli gibi davranır.

Mars Pathfinder (1997): NASA rover'ında düşük öncelikli meteoroloji thread'i mutex tutarken orta öncelikli thread onu geride bıraktı; yüksek öncelikli thread mutex'i alamadı ve sistem resetlendi.

Çözüm	Açıklama
Kesme Devre Dışı	Kritik bölgede tüm kesmeler kapatılır (kullanıcı için uygun değil)
Öncelik Tavanı (Priority Ceiling)	Mutex'e bir tavan öncelik atanır; mutex tutan thread bu önceliği alır
Öncelik Kalıtımı (Priority Inheritance)	Mutex bekleyen yüksek öncelikli thread'in önceliği, tutan thread'e geçici aktarılır (Pathfinder'da kullanıldı)
Rastgele Artırım (Windows)	Mutex tutan thread'lere rastgele yüksek öncelik verilir

3.11 Kilitless Okuma: RCU (Read-Copy-Update)

Kilit olmadan eş zamanlı okuma ve yazma: her okuyucu ya eski sürümü ya yeni sürümü görür, karışık sürüm görmez. Linux çekirdeğinde binlerce kullanımı vardır (ağ yığını, dosya sistemi, bellek yönetimi).

Temel fikir: Yeni düğüm tam hazırlandıktan sonra atomik bir yazmayla ağaca bağlanır. Eski düğüm kaldırılınca, tüm okuyucuların o düğümden ayrıldığı garanti edilene kadar beklenir (grace period). Sonra bellek serbest bırakılır.

4. Zamanlama (Scheduling)

Birden fazla süreç CPU için yarışırken hangi sürecin ne kadar çalışacağına **zamanlayıcı (scheduler)** karar verir; kullandığı algoritmaya ise **zamanlama algoritması (scheduling algorithm)** denir.

Ortam	Hedef
Toplu (Batch)	Verimlilik: İş çıktısı/saat, dönüşüm süresi, CPU kullanımı
Etkileşimli (Interactive)	Yanıt süresi, orantılılık (kullanıcı beklentisi)
Gerçek Zamanlı (Real-Time)	Son teslim tarihleri, öngörülebilirlik

4.1 Süreç Davranışı

Süreçler **CPU patlamaları** ve **G/Ç beklemleri** döngüsünü tekrar eder:

- **CPU-bound (hesaplama ağırlıklı):** Uzun CPU patlamaları, seyrek G/Ç.
- **I/O-bound (G/Ç ağırlıklı):** Kısa CPU patlamaları, sık G/Ç. CPU'ları hızlandıkça daha yaygın hale gelir.

4.2 Zamanlama Kararı Anları

- Yeni süreç oluşturulduğunda (ebeveyn mi, çocuk mu?)
- Süreç sonlandığında (sıradaki hangisi?)
- Süreç G/Ç, semafor vb. için bloklandığında
- G/Ç kesmesi olduğunda (G/Ç bekleyen süreç hazır olabilir)

Tür	Açıklama
Önceliksiz (Non-preemptive)	Süreç kendi iradesiyle CPU bırakana kadar çalışır. Saat kesmesi etkisiz.
Öncelikli (Preemptive)	Kuantum süresi dolunca CPU alınır. Zaman paylaşım için zorunlu.

4.3 Toplu Sistem Algoritmaları

FCFS — İlk Gelen İlk Çalışır

Tek kuyruk, öncelik yok. Uygulaması en basit algoritmadır. Dezavantaj: I/O-bound süreçler uzun CPU-bound bir süreç nedeniyle çok uzun bekleyebilir (konvoy etkisi).

SJF — En Kısa İş Önce

Tüm işler aynı anda varsa en düşük ortalama dönüşüm süresini verir (kanıtlanmış optimal). Çalışma süreleri önceden bilinmeli.

Örnek: Süre: 8, 4, 4, 4. FCFS → ort. 14 dk; SJF → 4, 8, 12, 20 → ort. 11 dk.

SRTN — En Kısa Kalan Süre Önce

SJF'nin öncelikli versiyonu. Yeni gelen iş, mevcut işin kalan süresinden daha kısaysa mevcut iş durdurulur, yeni iş çalıştırılır.

4.4 Etkileşimli Sistem Algoritmaları

Round-Robin (Dairesel Zamanlama)

Her süreç bir **kuantum** (zaman dilimi) çalışır; süresi dolunca sona alınır. Kuantum seçimi kritiktir:

Kuantum	Etki
Çok kısa (< 1 ms)	Aşırı bağlam geçişi → CPU verimsiz
Çok uzun (> 1 sn)	Yavaş yanıt → FCFS'e dönüşür
20-50 ms	İyi uzlaşma

Öncelik Zamanlaması (Priority Scheduling)

Her sürecin bir önceliği vardır; en yüksek öncelikli çalışır. Dinamik öncelik: CPU'yu az kullanan sürece yüksek öncelik (I/O-bound tercih edilir). Sınıflar içinde Round-Robin kullanılabilir.

Çoklu Kuyruk (Multiple Queues — CTSS)

Öncelik sınıfları: 1. sınıf 1 kuantum, 2. sınıf 2 kuantum, 3. sınıf 4 kuantum... Kuantumunu bitiren süreç bir alt sınıfa düşer. Kısa etkileşimli süreçler üst sınıflarda kalır, uzun hesaplama işleri yavaşça aşağı iner.

Lotarya Zamanlaması (Lottery Scheduling)

Her süreç bilet alır; her zamanlama kararında rastgele bilet çekilir. Önemli süreçlere fazla bilet → orantılı CPU payı. İşbirlikçi süreçler bilet devredebilir.

Adil Hisse Zamanlaması (Fair-Share)

Kullanıcı başına CPU payı garantilenir. Bir kullanıcı 9, diğeri 1 süreç çalıştırsa bile her kullanıcı vaat edilen payını alır.

En Kısa Süreç Önce / Yaşlandırma

Geçmiş davranışa dayanarak bir sonraki CPU patlaması tahmin edilir. **Yaşlandırma (aging)**
formülü: $S(n+1) = a \cdot S(n) + (1-a) \cdot T(n)$

4.5 Gerçek Zamanlı Zamanlama

m periyodik olay; i olayı P_i periyodunda C_i saniye CPU gerektirir. Sistemi zamanlanabilir kılmak için:
 $\sum (C_i/P_i) \leq 1$

Örnek: Periyot 100, 200, 500 ms; CPU: 50, 30, 100 ms → $0.5 + 0.15 + 0.2 = 0.85 \leq 1 \rightarrow$ zamanlanabilir.

4.6 Mekanizma - Politika Ayrımı

Zamanlama **mekanizması** çekirdekte bulunur (öncelik tabanlı algoritma). **Politika** ise kullanıcı süreçleri tarafından belirlenir (sistem çağrısıyla çocuk süreçlerin öncelikleri ayarlanabilir). Bu ayırım işletim sistemi tasarımında önemli bir ilkedir.

5 Olay Tabanlı Sunucular (Event-Driven / FSM)

Thread kullanılmadığında ya da istenmediğinde üçüncü bir yaklaşım: **sonlu durum makinesi (finite-state machine)**. Tek thread, bloklayan olmayan (nonblocking) sistem çağrıları ve select()/epoll gibi G/Ç bildirim mekanizmaları kullanır.

Model	Paralellik	Sistem Çağrısı	Programlama
Çok thread'li	Evet	Bloklayan (kolay)	Orta
Tek thread'li	Hayır	Bloklayan	Kolay (ama yavaş)
Olay tabanlı (FSM)	Evet	Bloklayan olmayan	Zor

□ **nginx / C10k Problemi**

epoll (Linux) ve kqueue (FreeBSD) ile olay tabanlı sunucular 10.000+ eş zamanlı bağlantıyı tek thread'le yönetebilir.

Acık Uçlu Sınav Hazırlık Soruları ve Cevapları

Soru 1

Süreç (process) ile program (program) arasındaki farkı açıklayın. Aynı program iki kez çalıştırıldığında kaç süreç oluşur ve neden?

Cevap:

Program, disk üzerinde pasif olarak depolanan bir talimat dizisidir; çalışmaz, yalnızca bir tarif gibi varlığını sürdürür.

Süreç ise o programın çalışmakta olan bir örneğidir: anlık program sayacı, yazmaçlar, bellek içerikleri ve durum bilgisi ile birlikte aktif bir varlıktır.

Aynı program iki kez çalıştırıldığında iki ayrı süreç oluşur. Her biri kendi program sayacı ve değişkenlerine sahiptir; kod segmenti paylaşılsa bile bunlar kavramsal olarak birbirinden bağımsız süreçlerdir.

Soru 2

Bir süreç hangi üç durumda bulunabilir? Hangi olaylar bu durumlar arasında geçişe neden olur? Tüm geçişleri (4 adet) açıklayın.

Cevap:

Running (Çalışıyor), Ready (Hazır) ve Blocked (Engellenmiş/Uyuyor) olmak üzere üç durum vardır.

Geçiş 1 — Running → Blocked: Süreç G/Ç için bloklanır (ör. read() çağırısı). Geçiş 2 — Running → Ready: Zamanlayıcı kuantum süresini doldurduğu için süreci keser. Geçiş 3 — Ready → Running: Zamanlayıcı bu süreci seçer. Geçiş 4 — Blocked → Ready: Beklenen olay gerçekleşir (G/Ç tamamlanır).

Not: Blocked → Running geçişi doğrudan olmaz; önce Ready, ardından Running'e geçilir.

Soru 3

Yarış koşulu (race condition) nedir? Yazdırma kuyruğu (spooler) örneği üzerinden açıklayın. Bu sorunu çözmek için ne gereklidir?

Cevap:

Yarış koşulu, iki veya daha fazla sürecin paylaşılan veriye eş zamanlı erişmesi ve sonucun çalışma sırasına bağımlı olması durumudur.

Spooler örneği: A süreci 'in' değişkenini 7 olarak okur; zamanlayıcı B'ye geçer. B de 7 okur, dosyasını 7. yuvaya yazar, in = 8 yapar. A tekrar çalışır, kendi dosyasını yine 7. yuvaya yazar → B'nin dosyası kaybolur.

Çözüm: Kritik bölgeye aynı anda yalnızca bir sürecin erişmesini sağlayan karşılıklı dışlama (mutual exclusion) mekanizması gereklidir.

Soru 4

Peterson çözümünü açıklayın. Hangi verileri kullanır ve dört kritik bölge koşulunun tümünü sağlar mı?

Cevap:

Peterson çözümü: turn (sıra değişkeni) ve interested[2] dizisi kullanılır. enter_region(proc): interested[proc]=1, turn=proc yazar; while(turn==proc && interested[other]) döngüsünde bekler.

leave_region(proc): interested[proc]=0 yazar.

Dört koşul sağlanır: (1) Aynı anda iki süreç kritik bölgede olamaz (turn kimin önce bıraktığını belirler). (2) CPU hızı varsayımı yok. (3) Kritik bölge dışı süreç başkasını engelleyemez. (4) Sonlu bekleme: diğer süreç leave_region çağırır ve interested=0 yapar.

Soru 5

Semafor nedir? down() ve up() işlemlerini açıklayın. Üretici-tüketici probleminde kaç semafor kullanılır ve neden?

Cevap:

Semafor, gelecekte kullanılmak üzere saklanan uyandırma sinyallerini tutan tamsayı değişkenidir.

down(s): $s > 0$ ise $s--$, devam et; $s = 0$ ise blokla. up(s): $s++$; bekleyen süreç varsa birini uyandır. Her iki işlem atomiktir.

Üretici-tüketici'de 3 semafor: mutex (karşılıklı dışlama, başlangıç: 1), empty (boş yuva sayısı, başlangıç: N), full (dolu yuva sayısı, başlangıç: 0). mutex kritik bölgeyi, empty/full ise tampon durumunu yönetir.

Soru 6

User-level thread ile kernel-level thread arasındaki farkları karşılaştırın. Her birinin en önemli bir avantajı ve dezavantajı nedir?

Cevap:

User-level thread: Kernel thread varlığından habersizdir. Geçişler hızlıdır (kernel çağrısı yok). Avantaj: Hız. Dezavantaj: Bloklayan sistem çağrısı tüm süreci durdurur.

Kernel-level thread: Kernel her thread'i bilir ve zamanlayabilir. Avantaj: Bir thread bloklarsa diğerleri çalışır. Dezavantaj: Thread geçişi için sistem çağrısı gerekir → yavaştır.

Soru 7

Round-robin zamanlamada kuantum uzunluğu nasıl seçilmelidir? Çok kısa ve çok uzun kuantumun etkileri nelerdir?

Cevap:

Çok kısa kuantum (örn. 1 ms): Bağlam geçişi süresi (1 ms) ile kıyaslanırsa CPU zamanının büyük kısmı bağlam geçişine harcanır (%50 israf). Çok uzun kuantum (örn. 1 sn): Kısa etkileşimli süreçler çok bekler; yanıt süresi kabul edilemez düzeyde artar.

Pratik uzlaşma: 20-50 ms aralığı, bağlam geçişi yükü ile yanıt süresi arasında iyi bir denge sağlar.

Soru 8

Öncelik tersine dönmesi (priority inversion) nedir? Mars Pathfinder örneğini açıklayın ve üç çözüm yöntemini belirtin.

Cevap:

Düşük öncelikli L süreci mutex tutarken yüksek öncelikli H süreci o mutex'i beklerse, orta öncelikli M süreci sürekli çalışır. H, L'nin mutex'i bırakmasını bekler; L ise M yüzünden çalışamaz → H sonsuza bekler.

Mars Pathfinder (1997): Meteoroloji thread'i (düşük) mutex tutarken haberleşme thread'i (orta) onu geride bıraktı; bilgi veri yolu thread'i (yüksek) mutex'i alamadı → sistem resetleri.

Çözümler: (1) Öncelik kalıtımı (priority inheritance) — mutex tutan thread, bekleyenin önceliğini geçici alır; Pathfinder'da kullanıldı. (2) Öncelik tavanı — mutex'e sabit tavan öncelik. (3) Rastgele artırım — Windows'ta kullanılır.

Soru 9

En Kısa İş Önce (SJF) algoritmasının optimal olduğunu kanıtlayın.

Cevap:

4 iş, çalışma süreleri a, b, c, d ($a \leq b \leq c \leq d$). 1. iş a 'da, 2. iş $a+b$ 'de, 3. iş $a+b+c$ 'de, 4. iş $a+b+c+d$ 'de biter.

Ortalama dönüşüm süresi = $(4a + 3b + 2c + d) / 4$.

a katsayısı en büyük (4) olduğundan a en küçük değer olmalıdır → a 'yı ilk çalıştır. Aynı argüman sırayla b, c, d için geçerlidir → artan sıralama optimaldir.

Not: Bu yalnızca tüm işler aynı anda varsa geçerlidir.

Soru 10

Kaybolan uyandırma (lost wakeup) problemi nedir ve semaforlar bu problemi nasıl çözer?

Cevap:

Sleep/wakeup ilkelerinde: Tüketici `count == 0`'ı görür, uyumak üzere iken zamanlayıcı üreticiye geçer. Üretici `wakeup()` gönderir; tüketici henüz uyumadığı için sinyal kaybolur. Tüketici uyur, üretici dolunca uyur → sonsuza kilitlenme.

Semafor çözümü: Semafor değeri, bekleyen uyandırmaları sayar. `down()` ve `up()` atomik olduğundan yarış oluşmaz. Tüketici `down(&full;)` çağırmadan önce üretici `up(&full;)` yaparsa değer 1 olur; tüketici `down()` çağırdığında hemen devam eder, uyumaz.

Kitap Bölüm Sonu Soruları ve Çözümleri

Problem 5

Bir bilgisayar sisteminin ana belleğine 4 program sığabiliyor. Bu programlar zamanlarının yarısında G/Ç bekliyor. CPU süresinin ne kadarı boşa gidiyor?

Çözüm:

$p = 0.5$ (G/Ç bekleme oranı), $n = 4$ (program sayısı).

Tüm 4 programın aynı anda G/Ç beklemesi olasılığı $= p^n = 0.5^4 = 0.0625$.

CPU kullanımı $= 1 - 0.0625 = 0.9375$ (yaklaşık %93.75).

Boşa giden CPU süresi $= \%6.25$.

Problem 6

2 GB RAM'e sahip bir bilgisayarda işletim sistemi 256 MB, her süreç 128 MB kullanıyor. Hedef %99 CPU kullanımı ise maksimum G/Ç bekleme süresi nedir?

Çözüm:

Kullanılabilir RAM: $2048 - 256 = 1792$ MB. Süreç sayısı $n = \text{floor}(1792/128) = 14$.

CPU kullanımı $= 1 - p^n = 0.99 \rightarrow p^{14} = 0.01 \rightarrow p = 0.01^{(1/14)}$.

$p = 0.01^{(1/14)} = e^{(\ln(0.01)/14)} = e^{(-0.329)} \approx 0.719$.

Sonuç: Maksimum G/Ç bekleme oranı yaklaşık %71.9'dur.

Problem 8

5 programlı bir çoklu programlama sisteminde her süreç zamanının %40'ında G/Ç bekliyor. CPU kullanımı ne olur?

Çözüm:

$p = 0.4$, $n = 5$.

CPU kullanımı $= 1 - p^n = 1 - 0.4^5 = 1 - 0.01024 = 0.98976$.

Yaklaşık %99 CPU kullanımı sağlanır.

Problem 16

Bir dosya sunucusu tek thread'li ise saniyede kaç istek işler? Çok thread'li ise? (Yönetim: 15 ms; disk operasyonu gerekirse +75 ms; 1/3 ihtimalle disk gerekir.)

Çözüm:

Ortalama istek süresi $= 15 + (1/3)*75 = 15 + 25 = 40$ ms.

Tek thread'li: 1 saniyede $1000/40 = 25$ istek/s.

Çok thread'li: Disk G/Ç süresinde (75 ms) başka thread'ler çalışır. Disk gerektiren thread bloklanır ama sunucu durmaz. Teorik kapasite: Disk olmayan istekler için $1000/15 \approx 67$ istek/s; disk isteklerinde G/Ç ve CPU örtüşür $\rightarrow$ yaklaşık 33 istek/s iyileşme. Pratik: Çok thread'li verimi önemli ölçüde artırır.

Problem 35

CDC 6600'de işlemci paylaşımı kullanılırken n süreç eş zamanlı çalışıyor. Bir süreç rekabetsiz ortamda T saniyelik işi bitirir. İşlemci paylaşımında ne kadar süre gerekir?

Çözüm:

İşlemci paylaşımında (round-robin, her talimat farklı süreç) CPU, n süreç arasında eşit bölünür. Her süreç efektif olarak n kat yavaş çalışır.

Gerekli süre $= n * T$.

Problem 41

Ortalama süreç G/Ç öncesi T süre çalışıyor; bağlam geçişi S süre alıyor. Round-robin'de Q kuantum için CPU verimliliği formülleri (a-e):

Çözüm:

- (a) $Q = \infty$: Süreç G/Ç'ye kadar çalışır, geçiş sadece bloklanınca. Verimlilik = $T / (T + S)$.
- (b) $Q > T$: Her çalışmada bir geçiş olur (süreç G/Ç bekler). Verimlilik = $T / (T + S)$.
- (c) $S < Q < T$: Her Q'da bir geçiş. Verimlilik = $Q / (Q + S)$.
- (d) $Q = S$: Verimlilik = $Q / (Q + S) = S / 2S = 1/2$ (%50).
- (e) $Q \rightarrow 0$: Verimlilik $\rightarrow 0$ (neredeyse tüm zaman bağlam geçişine gider).

Problem 43

A(10), B(6), C(2), D(4), E(8) — toplu işler. Öncelikler: 3,5,2,1,4. Round-robin, öncelik, FCFS, SJF için ortalama dönüşüm süresi:

Çözüm:

- (a) Round-robin: Her iş eşit CPU payı alır. Toplam iş = 30 dk. Her iş kendi süresini $30/5 = 6$ 'ya bölünmüş oranda alır (basit model). Tüm işler 30 dk'da tamamlanır $\rightarrow$ ortalama dönüşüm ≈ 30 dk (tüm işler eş zamanlı biter).
- (b) Öncelik zamanlaması (B=5, E=4, A=3, C=2, D=1 sırasıyla): B:6, E:8 $\rightarrow$ 14, A:10 $\rightarrow$ 24, C:2 $\rightarrow$ 26, D:4 $\rightarrow$ 30. Dönüşümler: 6,14,24,26,30. Ortalama = $(6+14+24+26+30)/5 = 100/5 = 20$ dk.
- (c) FCFS (A,B,C,D,E sırasıyla): 10, 16, 18, 22, 30. Ortalama = $(10+16+18+22+30)/5 = 96/5 = 19.2$ dk.
- (d) SJF (C=2, D=4, B=6, E=8, A=10): 2, 6, 12, 20, 30. Ortalama = $(2+6+12+20+30)/5 = 70/5 = 14$ dk. (En iyi sonuç)

Problem 46

Gerçek zamanlı sistem: 2 ses çağrısı (periyot 5 ms, CPU 1 ms) + 1 video akışı (periyot 33 ms, CPU 11 ms). Zamanlanabilir mi?

Çözüm:

Toplam yük = $C1/P1 + C2/P2 + C3/P3 = 1/5 + 1/5 + 11/33$.
 $= 0.2 + 0.2 + 0.333 = 0.733$.
 $0.733 \leq 1$ olduğundan sistem zamanlanabildir.

Problem 48

Yaşlandırma algoritması $a=1/2$ ile önceki 4 çalışma: 40, 20, 40, 15 ms. Bir sonraki tahmin nedir?

Çözüm:

$T_0=40, T_1=20, T_2=40, T_3=15$. $S(1) = (1/2)*40 + (1/2)*20 = 30$.
 $S(2) = (1/2)*30 + (1/2)*40 = 35$.
 $S(3) = (1/2)*35 + (1/2)*15 = 25$.
Bir sonraki tahmin = 25 ms.

Problem 49

4 periyodik olay: periyotlar 50, 100, 200, 250 ms; CPU süreleri 35, 20, 10, x ms. x'in alabileceği maksimum değer nedir?

Çözüm:

$\sum(C_i/P_i) \leq 1$ koşulu: $35/50 + 20/100 + 10/200 + x/250 \leq 1$.
 $0.7 + 0.2 + 0.05 + x/250 \leq 1$.
 $0.95 + x/250 \leq 1 \rightarrow x/250 \leq 0.05 \rightarrow x \leq 12.5$ ms.
x'in maksimum değeri 12.5 ms'dir.

Bu belge, Tanenbaum — Modern Operating Systems kitabının Bölüm 2 içeriğine dayanılarak Türkçe olarak hazırlanmıştır.